



Anatomy of PBIP

💡 **What is PBIP?** Power BI Project format (`.pbip`) unpacks your entire Power BI report into a folder of human-readable text files — enabling Git version control, team collaboration, and CI/CD pipelines. This replaces the opaque `.pbix` binary blob.



The Big Picture

When you save a Power BI file in PBIP format, you get a **folder structure** — not a single file. Everything is split into logical, editable pieces.

```
MyReport/                                ← root project folder
|
├─ MyReport.Report/                      ← everything the user SEES
├─ MyReport.SemanticModel/              ← all DATA, tables, DAX
├─ .gitignore                           ← Git configuration
└─ MyReport.pbip                        ← launcher / entry point
```

🔑 **Key Insight:** The report (visuals) and the data model are completely **separated** into two independent folders. A data engineer can work on the model while a report developer works on visuals — simultaneously, without conflicts.



The `.pbip` File — The Launcher

This is just the **front door**. Double-clicking it opens your project in Power BI Desktop. It contains almost nothing — just a pointer to the `.Report` folder.

```
{
  "version": "1.0",
  "artifacts": [
    {
      "report": {
        "path": "MyReport.Report"
      }
    }
  ]
}
```

 Think of `.pbip` like the shortcut on your desktop. The actual work lives in the folders.

The `.Report` Folder — Everything the User Sees

This folder contains all **visual and layout** information. It knows nothing about your data — it only describes *what to show* and *how it looks*.

```

MyReport.Report/
|
├── report.json           ← report-level theme, filters, settings
├── definition.pbir       ← pointer to which Semantic Model to use
├── pages/               ← one sub-folder per report page/tab
|   ├── Page 1/
|   │   ├── page.json     ← page layout, background, size
|   │   └── visuals/      ← every chart, card, slicer on this page
|   └── Page 2/

```



▼ report.json — what's inside?

`report.json` contains top-level report settings:

- The **theme** applied to the whole report
- **Report-level filters** (filters that apply across all pages)
- Page order and names
- Report metadata

Open it in VS Code and you'll see clean, readable JSON. Every formatting decision you make in Power BI Desktop is stored here as text.

▼ definition.pbir — what's inside?

This is a **connection pointer** — it tells Power BI which Semantic Model to connect this report to.

```
{
  "version": "1.0",
  "datasetReference": {
    "byPath": {
      "path": "../MyReport.SemanticModel"
    }
  }
}
```

This separation is powerful — you can point **multiple different reports** at the **same semantic model**. One model, many faces.

▼ pages/ folder — what's inside?

Each report page (tab) gets its own sub-folder. Inside that folder:

- **page.json** — the page's canvas size, background color, display name, and page-level filters
- **visuals/** — a folder where each visual (chart, card, table, slicer) is stored as its own JSON file describing its type, position, size, fields, and formatting

This means if you add a bar chart, only the visuals folder changes. If you rename a page, only page.json changes. Git can track all of this precisely.

The **.SemanticModel** Folder — The Brain

This folder contains your entire **data model**. Tables, columns, DAX measures, relationships, data source connections, security roles — all of it.

```

MyReport.SemanticModel/
├── definition.pbism      ← declares which model format is used
├── .platform            ← Power BI Service deployment metadata
├── model.bim            ← (OPTION A) entire model as one JSON file
└── definition/          ← (OPTION B) modern TMDL format
    ├── database.tmdl    ← model-level settings
    ├── relationships.tmdl ← all table relationships defined here
    └── tables/
        ├── Sales.tmdl   ← Sales table: columns, measures, formatting
        ├── Calendar.tmdl ← Calendar table
        └── Products.tmdl ← Products table

```

⚡ **TMDL vs model.bim:** You have two format choices for storing your data model. TMDL (modern) splits it into one file per table. model.bim (legacy) keeps everything in one giant JSON. TMDL is far better for Git — only the changed table's file is modified.

▼ What does a TMDL file actually look like?

TMDL = **Tabular Model Definition Language**. It's designed to be human-readable. Here's what a `Sales.tmdl` file looks like:

```
table Sales

    column OrderID
        dataType: int64
        isHidden: true

    column SalesAmount
        dataType: decimal
        formatString: $#,##0.00

    column OrderDate
        dataType: dateTime
        formatString: Short Date

    measure [Total Sales] =
        SUM(Sales[SalesAmount])
        formatString: $#,##0
        displayFolder: Revenue Metrics

    measure [YTD Sales] =
        TOTALYTD([Total Sales], 'Calendar'[Date])
        formatString: $#,##0
        displayFolder: Revenue Metrics
```

Every table, column, and DAX measure — written in plain text that you can read, edit, and review in a pull request.

▼ What is the .platform file?

The `.platform` file stores metadata used when deploying to **Power BI Service** (the online version). It contains things like:

- The item type (e.g. `SemanticModel`)
- The display name in the Service
- Version compatibility info

You rarely need to edit this manually — Power BI manages it for you.

The `.gitignore` File — The Unsung Hero

This file tells Git which files to **skip tracking**. Power BI generates some temporary and cache files that are local-only and would cause conflicts if everyone committed them.

You don't need to edit this file. Power BI pre-configures it sensibly. Just know it exists to keep your Git history clean — only meaningful changes get tracked.

Typical ignored items:

- Local cache folders (`.pbi/`)
 - Lock files
 - Temporary user-specific settings
-

Report vs SemanticModel — The Separation That Matters

	 <code>.Report</code> folder	 <code>.SemanticModel</code> folder
Contains	Charts, pages, visuals, themes	Tables, columns, DAX, relationships
Changed by	Report developers / UX designers	Data engineers / analysts

	 .Report folder	 .SemanticModel folder
Knows about data?	✗ No — it just references fields by name	✓ Yes — this is the data layer
Can be reused?	One report = one dataset by default	One model → many reports
Git changes when	You add/move/resize a visual	You write a new DAX measure

 **Real-world power:** In a team, the data engineer pushes changes to `SemanticModel/` and the report developer pushes changes to `Report/`. Git merges both cleanly. No more emailing `.pbix` files back and forth.

PBIP vs PBIX — The Complete Comparison

Feature	 .pbix (old)	 .pbip (new)
File format	Binary blob (opaque)	Human-readable JSON/text
Git-friendly	✗ Can't see what changed	✓ Every change is visible
Team collaboration	Painful — one person at a time	✓ Designed for teams
Open in text editor	✗ No	✓ Yes — VS Code works great
Report/model separated	✗ Merged together	✓ Completely independent
Imported data stored	✓ Yes, inside the file	✗ No — always live connection
CI/CD pipelines	Very hard to set up	✓ Native support
Code reviews (PRs)	Impossible	✓ Full diff visibility
Multiple reports, one model	Requires separate setup	✓ Built-in via <code>definition.pbir</code>

Important Caveats for Beginners

Data is NOT stored in PBIP. Unlike `.pbix` which could embed imported data tables inside the file, PBIP always connects live to your data sources. You need an active data source connection to work with your model.

PBIP is a developer feature. It's designed for people who want Git, CI/CD, or team workflows. For simple personal reports, `.pbix` is still perfectly fine.

You can convert back and forth. Power BI Desktop lets you open `.pbip` and save as `.pbix`, and vice versa. You're not locked in.

How It All Connects

